AdaptiveKV: Attention-Guided Mixed-Precision KV Cache

[team member names] · Guide: [project guide] · [college / department]

Progress report for the project coordinator · 2026-10-22 · all numbers measured on an RTX 3050 Laptop (4 GB) and generated from results/ by benchmarks/make_report_pdf.py

In one sentence: choosing each cached token's precision by the attention it receives keeps the compressed KV cache closer to the full-precision model than the standard recency rule at the same memory (KL 0.0314 vs 0.0454 at 4.92 / 4.92 bits, p = 0.003). Our default two-tier cache (INT8/INT4, 50/50) stores 7.41 bits per value instead of 16 (2.16x smaller) at +0.1% perplexity.

What we built

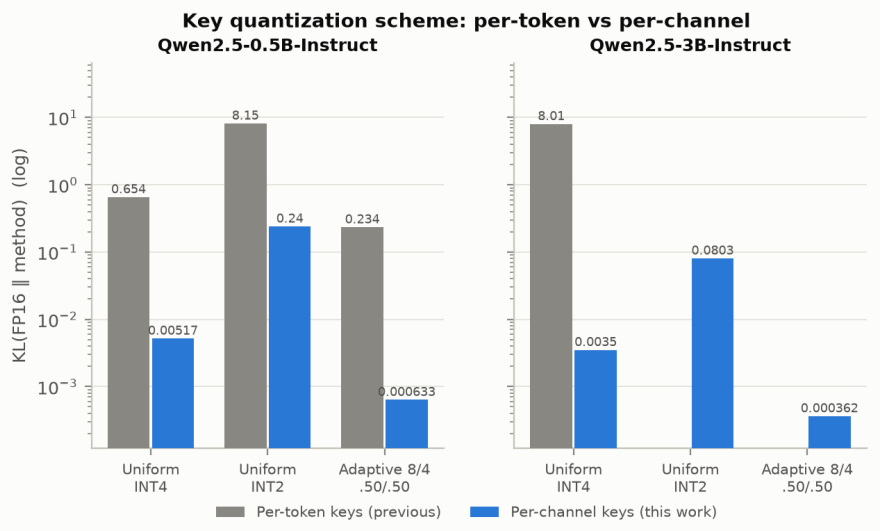
• A KV cache that keeps every cached token at INT8 or INT4 (INT2 tested as a third tier) and re-ranks tokens every step by an exponential moving average of the attention they receive. Tokens can only move down a tier, never up: once a token is stored at lower precision its exact values are gone.
• Per-channel key quantization: keys are quantized per channel over blocks of 32 tokens (values per token). This is the change that made low precision usable.
• Honest measurement: memory = real stored bytes including every scale and index; quality = perplexity and KL divergence measured through the compressed cache against FP16, 20 WikiText-2 windows, 95% confidence intervals and paired significance tests.

Setup: Qwen2.5-0.5B-Instruct fully on GPU (20 windows of 512 prefilled + 64 scored tokens); Qwen2.5-3B-Instruct with weights partly offloaded to CPU (3 windows, a confirmation only). Pure PyTorch, no custom GPU kernels.

Headline numbers (Qwen2.5-0.5B, lower bits / KL / ΔPPL = better)

Configuration	Bits/value	Smaller than FP16	Perplexity change	KL to FP16
fp16	16.00	1.00x	+0.0%	0.0000
int8 (uniform)	8.99	1.78x	+0.0%	0.0000
int4 (uniform)	5.13	3.12x	+0.6%	0.0052
int4 (uniform) - OLD key scheme	4.52	3.54x	+88.5%	0.6545
int2 (uniform)	3.20	5.01x	+22.5%	0.2401
int2 (uniform) - OLD key scheme	2.52	6.36x	+300253.1%	8.1540
adaptive 8/4 0.50/0.50 (default)	7.41	2.16x	+0.1%	0.0006
recency 8/4 0.50/0.50	7.08	2.26x	-0.0%	0.0009
adaptive 8/4/2 0.10/0.40/0.50	4.92	3.25x	+3.1%	0.0314
recency 8/4/2 0.20/0.30/0.50	4.92	3.25x	+2.2%	0.0454

Shaded rows: our two main results. The default is two-tier INT8/INT4 at 50/50; recency at the same fractions is shown below it. The three-tier 8/4/2 rows are the INT2 experiment: there, attention-guided and recency use the same memory (4.92 / 4.92 bits) and adaptive is closer to FP16, but no INT2 mix beat uniform INT4 (5.13 bits, +0.6%), so INT2 is reported as a negative result (check 15, page 3). "OLD key scheme" = the previous design (per-token keys).

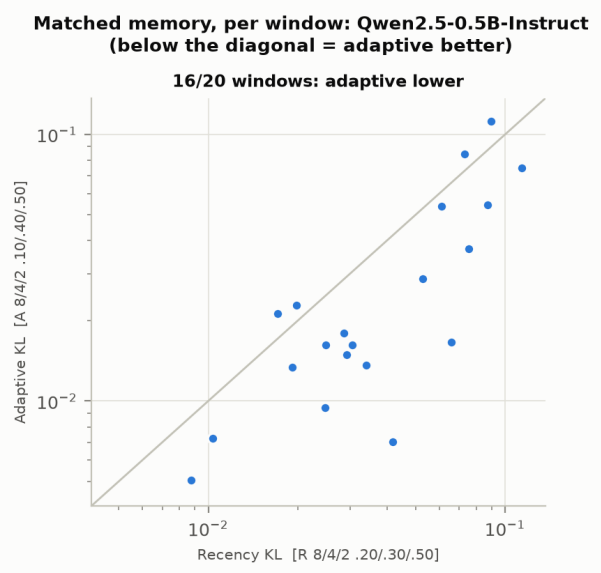


On Qwen2.5-3B only uniform INT4 was run with the old key scheme, so the other two 3B bars have no grey partner.

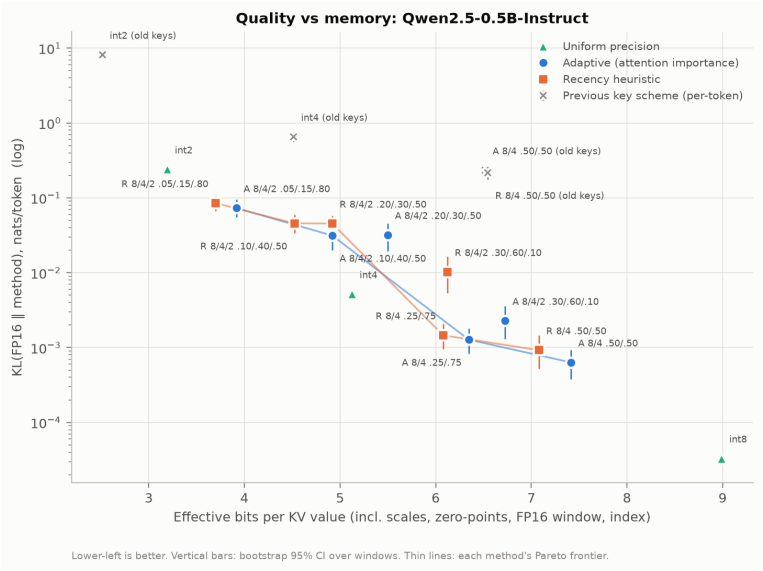
What we found: three results

1. Keys need per-channel quantization. A few key channels carry very large values. Quantizing each token separately lets those channels stretch every token's range, so all other channels lose precision. Quantizing each channel across tokens gives every channel its own range. Result: INT4 KL to FP16 drops from 0.654 to 0.0052 (127x lower) and perplexity from +88.5% to +0.6%. On the larger Qwen2.5-3B the old scheme breaks the model (+263,677% perplexity) while ours gives +1.1%.

2. Attention tells us which tokens matter. At the same memory (4.92 / 4.92 bits), the attention-guided cache is closer to the full-precision model than the recency rule in 16 of 20 text windows: KL 0.0314 vs 0.0454 (difference -0.0140, 95% CI [-0.0219, -0.0061], p = 0.003) and top-1 agreement with FP16 93.1% vs 90.2% (p = 0.006). Each dot below is one text window; dots under the diagonal are windows where we win.



3. The reason it works is measurable. We checked whether attention importance predicts the attention a token gets later. When 25% of tokens are kept at INT8, attention importance keeps 58% of the future attention there, against 51% for recency. When 50% of tokens are kept at INT8, attention importance keeps 79% of the future attention there, against 70% for recency. (Wilcoxon p = 1.91e-06 in both cases; re-run on the current code on 2026-10-22.) More of the attention the model will actually use stays at high precision.



The gap between attention-guided and recency is largest below about 5.5 bits, where INT2 is used; at 6 bits and above the two are close. The equal-memory significance test is in result 2.

Limitations

• Only stored KV bytes shrink: the cache is dequantized to FP16 for every attention call, so peak GPU memory is unchanged and decoding is slower. Fused dequantize-attention kernels would be needed for both.
• Uniform INT4 with per-channel keys is already near-lossless; no INT2 mix beat it at equal or fewer bits. The attention advantage is shown against recency when precisions are mixed.
• Quality is measured on WikiText-2 and a synthetic passkey task only; 3B results use few windows.
Full details: results/PAPER_REPORT.md (paper draft) and results/VERIFICATION.md (every test and number).

Test report

Automated test suite: 71 passed, 0 failed, 0 errors, 0 skipped (run 2026-10-13 17:24). Unit tests check quantization, packing, tier quotas, memory accounting by hand, cache ordering; integration tests run the real model on the GPU.

Test file	What it checks	Passed	Failed
test_allocator	tier quotas, pins	8	0
test_attention_capture	OOM regression (2048 tokens)	2	0
test_cache	previous cache + byte accounting	7	0
test_cache_channel	per-channel key cache (new)	10	0
test_importance	attention hook + EMA	6	0
test_int2_ablation	three-tier (INT2) logic	14	0
test_integration	real model on GPU	10	0
test_paper_metrics	KL, CIs, Pareto, safety guard	9	0
test_quant	INT8/4/2 quantize + bit packing	5	0

Experiment checks (claims tested against the data)

Each claim has a fixed pass threshold. A FAIL is a real negative result, reported as measured.

#	Claim	Result	Evidence
1	INT8 cache is near-lossless (perplexity change below 0.1%)	**PASS**	+0.019%
2	Per-channel keys cut INT4 error at least 10x vs old scheme (0.5B)	**PASS**	KL 0.654 → 0.0052 (127x)
3	Per-channel keys cut INT4 error at least 10x vs old scheme (3B)	**PASS**	KL 8.01 → 0.0035 (2290x)
4	INT4 and attention-guided caches retrieve the passkey as often as FP16	**PASS**	fp16: 60 / 60, int4: 60 / 60, adaptive 8/4/2 0.10/0.40/0.50: 60 / 60
5	INT4 cache within 1% perplexity of FP16 (0.5B)	**PASS**	+0.58% at 5.13 bits
6	INT4 cache within 2% perplexity of FP16 (3B)	**PASS**	+1.06%
7	Stored cache at least 3x smaller than FP16 (INT4)	**PASS**	3.12x
8	Every method runs at 2048 tokens without out-of-memory	**PASS**	5 methods, 0 OOM / 0 errors
9	Attention-guided beats recency on KL at equal memory (p<0.05)	**PASS**	Δ -0.0140, p = 0.003
10	Attention-guided beats recency on top-1 agreement (p<0.05)	**PASS**	Δ +0.029, p = 0.006
11	Importance predicts future attention better than recency (p<0.05)	**PASS**	p = 1.9e-06
12	Attention-guided beats recency on perplexity at equal memory (p<0.05)	**FAIL**	Δ NLL +0.0083, p = 0.39 (not significant)
13	Attention-guided beats recency on passkey retrieval (p<0.05)	**FAIL**	3 vs 0 wins, p = 0.25; not at equal memory: adaptive used 5.57 vs 4.82 bits (8/4/2 0.20/0.30/0.50) and 5.03 vs 4.42 bits (8/4/2 0.10/0.40/0.50)
14	Same KL advantage confirmed on 3B (p<0.05)	**FAIL**	Δ -0.0099, p = 0.25 (only 3 windows)
15	A three-tier (INT2) mix beats uniform INT4 at equal or fewer bits	**FAIL**	uniform INT4 already near-lossless
16	Peak GPU memory lower than FP16	**FAIL**	1.42 GB vs 1.42 GB (dequantized per step)
17	Decoding as fast as FP16	**FAIL**	70 vs 14 ms/token (no custom GPU kernel)

11 of 17 checks pass. The failures are expected limits of a software prototype (peak memory, speed) or effects too small to prove with this sample (perplexity, passkey, 3B with 3 windows). The main claims, compression with near-FP16 quality and the attention advantage on KL, pass.

Problems found during development and fixed

- INT4 broke the 3B model (old per-token keys) → fixed by per-channel keys.
- Out-of-memory at 2048 tokens (library kept every layer's attention) → importance read without storing it; regression test added; the no-out-of-memory check now passes.
- A test failed after the redesign (short prompts got no tier) → fixed in the design, test left unchanged.
- Reporting errors (crashes logged as out-of-memory; hard-coded wrong bytes-per-token) → fixed.